K.H.U. Faculty of Engineering

FENS 402 ENGINEERING DESIGN PROJECT
PROGRESS REPORT

MULTIMODAL MIXTURE OF EXPERTS (MOE) FOR VISUAL QUESTION ANSWERING (VQA)

Project Advisor: Assoc. Prof. Dr. Taner Arsan

Project Team: Mehmet Kuzucu[1], Şafak Gün[1]

Submission Date: 04.04.2025

[1] Computer Engineering, Engineering and Natural Sciences Faculty, Kadir Has University

PROGRESS:

OVERVIEW

This progress report presents a detailed update on our ongoing work in the field of Visual Question Answering (VQA). This project is motivated by the uprising need for AI systems to understand both images and text, especially for tasks that require reasoning using both types of information. VQA is actually a multimodal task, where a system is expected to answer natural language questions based on visual inputs. To push the boundaries of performance in VQA systems, our project aims to show the capabilities of a MoE-architectured LLM and a vision encoder by combining their strengths for better results.

In the early stages of the project, we conducted an extensive literature survey on VQA architectures, vision-language fusion strategies, and state-of-the-art LLMs. This included in-depth analysis of transformer-based architectures, vision transformers (ViTs), and the emerging class of mixture-of-experts models, particularly those optimized for low-resource environments. Since we don't have a high budget, we have to search for the methods that are suitable for low-resource environments. The insights gained from this exploration guided our architectural design and dataset selection strategies.

Our proposed system uses a novel cross-attention mechanism designed to fuse information extracted from images and text efficiently. We utilized pretrained models such as ViT for image encoding and LLMs like Qwen2-0.5B for processing text-based queries. A linear projection layer maps visual features into a shared latent space, which are then fused with textual embeddings using our custom cross-modal attention blocks. The model has been undergoing training and hyperparameter tuning, with promising first results.

DATA COLLECTION AND PREPROCESSING

As you know, the reliability and performance of any deep learning system are heavily reliant on the quality and diversity of the data it is trained on. Huge efforts were made to gather, clean, and organize large, high-quality datasets that cover many real-world VQA situations.

We selected several well-established datasets for text-based question-answering tasks, which are used both individually and together as we are still determining which one works best with our model:

- **SQuAD (Stanford Question Answering Dataset):** A benchmark dataset for machine reading comprehension. The details of the dataset can be found in Figure 1.

id string · *lengths* 24 24	title string · *classes* 442 values	context string · *lengths* 151 3.71k	question string · *lengths* 1 25.7k	answers sequence
5733be284776f41900661182	University_of_Notre_Dame	Architecturally, the school has a Catholic…	To whom did the Virgin Mary allegedly appear in…	{ "text": ["Saint Bernadette Soubirous"]…
5733be284776f4190066117f	University_of_Notre_Dame	Architecturally, the school has a Catholic…	What is in front of the Notre Dame Main Building?	{ "text": ["a copper statue of Christ"],…
5733be284776f41900661180	University_of_Notre_Dame	Architecturally, the school has a Catholic…	The Basilica of the Sacred heart at Notre Dame is…	{ "text": ["the Main Building"],…
5733be284776f41900661181	University_of_Notre_Dame	Architecturally, the school has a Catholic…	What is the Grotto at Notre Dame?	{ "text": ["a Marian place of prayer and…
5733be284776f4190066117e	University_of_Notre_Dame	Architecturally, the school has a Catholic…	What sits on top of the Main Building at Notre…	{ "text": ["a golden statue of the Virgin…
5733bf84d058e614000b61be	University_of_Notre_Dame	As at most other universities, Notre Dame'…	When did the Scholastic Magazine of Notre dame…	{ "text": ["September 1876"], "answer_start"…

Figure 1. Parameters of SQuAD Dataset

- **TriviaQA:** A collection of trivia questions with evidence documents from Wikipedia and web search. The details of the dataset can be found in Figure 2.

question string	question_id string	question_source string	entity_pages sequence	search_results sequence	answer dict
What is the distance between…	tc_150	http://www.triviacountry.com/	{ "doc_source": ["TagMe", "TagMe"], "filename": ["Baseball_field.txt", "L…	{ "description": ["Field Specificati ons. ... the distance between base p…	{ "aliases": ["60 distance", "60 feet", "sixty distance"],…
How many home runs did baseball grea…	tc_154	http://www.triviacountry.com/	{ "doc_source": ["TagMe", "TagMe", "Ta gMe"], "filename": ["Baseball.txt",…	{ "description": ["A 1942 survey of former major league managers pointed…	{ "aliases": ["None", "None (disambiguation)"],…
In cross-country bike racing, what…	tc_155	http://www.triviacountry.com/	{ "doc_source": [], "filename": [], "ti tle": [], "wiki_context": [] }	{ "description": ["In cross-country bike racing, what do the initials BM…	{ "aliases": ["Bicycle moto x (cross)"], "normalized_aliases": ["bicycle…
After retiring as a player, with…	tc_164	http://www.triviacountry.com/	{ "doc_source": [], "filename": [], "ti tle": [], "wiki_context": [] }	{ "description": ["Babe Ruth is a fo rmer Major League Baseball player,…	{ "aliases": ["The Brooklyn Dodgers in 1938"], "normalized_aliases": […
What is the maximum weight…	tc_166	http://www.triviacountry.com/	{ "doc_source": [], "filename": [], "ti tle": [], "wiki_context": [] }	{ "description": ["The Trailblazer: Jackie Robinson By Larry Canale The …	{ "aliases": ["350 pounds. The minimum is 200 pounds"], "normalized_aliases"…
What baseball player hit the…	tc_168	http://www.triviacountry.com/	{ "doc_source": [], "filename": [], "ti tle": [], "wiki_context": [] }	{ "description": ["What Baseball Hal l of Fame pitcher hit a home run in …	{ "aliases": ["Joe Nickro in 1976. Nickro, a pitcher with the Houston…
Brooks Robinson and Carl…	tc_171	http://www.triviacountry.com/	{ "doc_source": [], "filename": [], "ti tle": [], "wiki_context": [] }	{ "description": ["Brooks Robinson a nd Carl Yastrzemski hold the major l…	{ "aliases": ["23 years. Third baseman Robinson played with the Baltimore…
Under the rules outlined in the…	tc_175	http://www.triviacountry.com/	{ "doc_source": [], "filename": [], "ti tle": [], "wiki_context": [] }	{ "description": ["Under the rules o utlined in the charter of the Intern…	{ "aliases": ["At least 6 grams. Silver medals must be at least .925…
Who was the famous great-great-…	tc_179	http://www.triviacountry.com/	{ "doc_source": [], "filename": [], "ti tle": [], "wiki_context": [] }	{ "description": ["Biography of Stev e Young, Quarterback of the San Fran…	{ "aliases": ["Mormon leader Brigham Young"], "normalized_aliases": […
How many of the four Grand Slam…	tc_184	http://www.triviacountry.com/	{ "doc_source": [], "filename": [], "ti tle": [], "wiki_context": [] }	{ "description": ["At least 6 grams. Silver medals must be at least .925 …	{ "aliases": ["Only the Wimbledon trophy is gold; the others--for the…
"What breakfast food gets its nam…	tc_186	http://www.triviacountry.com/	{ "doc_source": ["Search"], "filenam e": ["Bagel.txt"], "title": ["Bagel…	{ "description": ["The history of th e bagel, the familiar breakfast food…	{ "aliases": ["New York bagel", "American bagel", "Biegel", "Bagels",…
What popular drink did a Dutch…	tc_190	http://www.triviacountry.com/	{ "doc_source": [], "filename": [], "ti tle": [], "wiki_context": [] }	{ "description": ["... What popular drink did a Dutch medical professor …	{ "aliases": ["Mothers ruin", "List of gins", "List of gin brands", "London…
On what vegetable did an ancient…	tc_194	http://www.triviacountry.com/	{ "doc_source": [], "filename": [], "ti tle": [], "wiki_context": [] }	{ "description": ["On what vegetable did an ancient Egyptian place his ri…	{ "aliases": ["The onion. Its round shape symbolized eternity"],…

Figure 2. Parameters of TriviaQA Dataset

- **Databricks Datasets:** Used to enhance our model's general comprehension and reasoning capabilities. The details of the dataset can be found in Figure 3.

instruction string	context string	response string	category string
When did Virgin Australia start operating?	Virgin Australia, the trading name of Virgin Australia Airlines Pty Ltd, is an Australian-based…	Virgin Australia commenced services on 31 August 2000 as Virgin Blue, with two aircraft on a single route.	closed_qa
When was Tomoaki Komorida born?	Komorida was born in Kumamoto Prefecture on July 10, 1981. After graduating from high school, he joined…	Tomoaki Komorida was born on July 10,1981.	closed_qa
If I have more pieces at the time of stalemate, have I won?	Stalemate is a situation in chess where the player whose turn it is to move is not in check and has no…	No. Stalemate is a drawn position. It doesn't matter who has captured more pieces or is in a winning…	information_extraction
Given a reference text about Lollapalooza, where does it take place, who started it and what is it?	Lollapalooza /ˌlɒləpəˈluːzə/ (Lolla) is an annual American four-day music festival held in Grant Park…	Lollapalooze is an annual musical festival held in Grant Park in Chicago, Illinois. It was started in…	closed_qa
Who was John Moses Browning?	John Moses Browning (January 23, 1855 – November 26, 1926) was an American firearm designer who developed…	John Moses Browning is one of the most well-known designer of modern firearms. He started building…	information_extraction
Who is Thomas Jefferson?	Thomas Jefferson (April 13, 1743 – July 4, 1826) was an American statesman, diplomat, lawyer, architect,…	Thomas Jefferson (April 13, 1743 – July 4, 1826) was an American statesman, diplomat, lawyer, architect,…	information_extraction
Who was Kyle Van Zyl playing against when he scored 36 of hisa teams 61 points?	Van Zyl joined the Eastern Province Kings Academy, where he played for the Eastern Province U19 side in…	Kyle Van Zyl was playing against Boland U21 when he scored 36 points, leading his team to victory in a…	closed_qa
From the passage list down the areas for which Dar es Salaam is Tanzania's most prominent city. List the…	Dar es Salaam (/ˌdɑːr ɛs səˈlɑːm/; from Arabic: دار السلام, romanized: Dār es-Salām, lit.'Abode of…	arts, fashion, media, film, television, finance	information_extraction
What is process mining?	Process mining is a family of techniques relating the fields of data science and process management to…	Process mining brings together data mining, data science and process management to find process…	information_extraction
What is a dispersive prism?	In optics, a dispersive prism is an optical prism that is used to disperse light, that is, to separate…	A dispersive prism is an optical prism that disperses the light's different wavelengths at different…	summarization
Extract the owner of Lamborghini and a listing of the different types of Huracan cars that Lamborghini has…	Automobili Lamborghini S.p.A. (Italian pronunciation: [autoˈmɔːbili lamborˈgiːni]) is an Italian…	Vokswagen Group owns Lamborghini through its subsidiary, Audi. Lamborghini has produced the…	information_extraction
What is the average lifespan of a Golden Retriever?	Golden Retrievers are a generally healthy breed; they have an average lifespan of 12 to 13 years.…	The average lifespan of a Golden Retriever is 12 to 13 years.	information_extraction
Please summarize what Linkedin does.	LinkedIn (/lɪŋktˈɪn/) is a business and employment-focused social media platform that works through…	Linkedin is a social platform that business professionals create profiles on and network with…	summarization
Using examples taken from the text give me a summary of the main arguments in favour of slavery…	Slavery ended in the United States in 1865 with the end of the American Civil War and the ratification o…	In spite of progressive changes since the end of enforcement of Jim Crow and the successes of the…	summarization

Figure 3. Parameters of Databricks Dataset

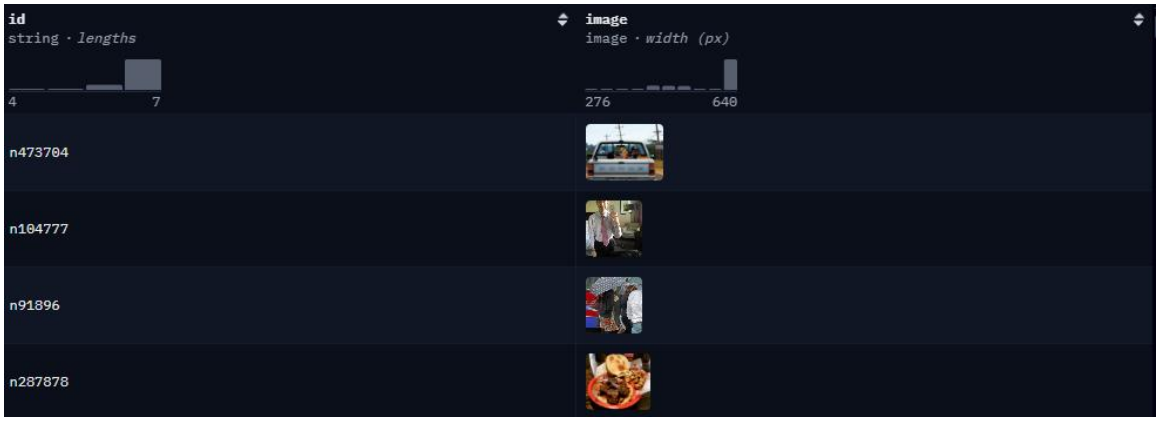
Image-based question answering was facilitated by the following datasets, which are used both individually and together too since we are still determining which one fits our model the best:

- **COCO (Common Objects in Context):** A rich dataset containing labeled images with natural object annotations. The details of the dataset can be examined in Figure 4.

image_id int64	image image · *width (px)*	width int64	height int64	objects sequence
9 160k	100 640	100 640	100 640	
9		640	480	{ "bbox_id": [1038967, 1039564, 1058555, 1534147, 1913551, 1913746, 1913856, 1914001], "category": …
25		640	426	{ "bbox_id": [598548, 599491], "category": [23, 23], "bbox": [[385.53, 60.03, 600.5,…
30		640	428	{ "bbox_id": [291613, 1155486], "category": [58, 75], "bbox": [[204.86, 31.02, 459.74, 355.14],…
34		640	425	{ "bbox_id": [589229], "category": [22], "bbox": [[0.96, 20.06, 442.19, 399.21]],…

Figure 4. Parameters of COCO Dataset

- **GQA (Graph Question Answering):** Designed specifically for VQA tasks with real-world scene understanding. The details of the dataset can be examined in Figure 5.



id string · *lengths*	image image · *width (px)*
4 7	276 640
n473704	
n104777	
n91896	
n287878	

Figure 5. Parameters of GQA Dataset

All datasets were carefully preprocessed, which included steps like text normalization, tokenization with pretrained tokenizer models, resizing and normalizing images, and removing irrelevant or noisy samples. For each dataset, we created custom dataset classes to handle sample loading.

MODEL DEVELOPMENT AND IMPLEMENTATION

We have been implementing and evaluating various architectures and we will select the best-performing ones. The models evaluated include:

Language Models: SmolLM2-135M, SmolLM-360M, GPT-2, Qwen2-0.5B. These models vary in size and architecture, offering diverse perspectives for answering language-based queries. We used "transformers" library for getting these different pre-trained LLMs. As an example of obtaining an LLM can be seen in Figure 6.

```
model_name = "HuggingFaceTB/SmolLM2-135M"
model = AutoModelForCausalLM.from_pretrained(model_name)
```

Figure 6. Obtaining SmolLM2-135M

Vision Encoders: We experimented with several encoders including ResNet-50, EfficientNet-B3, and ViT-Base. Ultimately, ViT-Base demonstrated superior performance due to its attention-based processing of image patches. How we utilized ViT-Base can be shown in Figure 7.

```
vit_model = vit_b_16(weights=ViT_B_16_Weights.IMAGENET1K_SWAG_LINEAR_V1)
```

Figure 7. Obtaining ViT-Base

We implemented a linear projection layer to transform vision features into the same embedding dimension as our LLMs. A custom cross-attention module was introduced to integrate visual and linguistic representations at multiple levels within the model. Training strategies included full training, mixed precision training, and LORA training method.

After getting the pre-trained models, LLMs, and image encoder, we have built our own Multimodal structure. Our image encoder class constists of transforming of the raw image and passing the image into encoder model. Our ImageEncoderVIT class demonstrates this process as can be seen in Figure 8.

```
class ImageEncoderVIT(nn.Module):
    def __init__(self):
        super().__init__()
        self.vit_model =
vit_b_16(weights=ViT_B_16_Weights.IMAGENET1K_SWAG_LINEAR_V1).to(device)
        self.vit_model.heads = nn.Identity()
        self.transform_func =
ViT_B_16_Weights.IMAGENET1K_SWAG_LINEAR_V1.transforms()
    def forward(self, x):
        x = self.transform_func(x)
        x = x.unsqueeze(0).to(device)
        x = self.vit_model(x)
        return x
```

Figure 8. Architecture of the ImageEncoderVIT Class

Our cross attention class consisting of cross attention block and normalization layers. In block image features will attend the text features which will provide us a fusion mechanism that can be used in the text generation as can be seen in Figure 9.

```
class CrossAttentionFusion(torch.nn.Module):
    def __init__(self, embed_dim, num_heads):
        super(CrossAttentionFusion, self).__init__()
        self.multihead_attention = torch.nn.MultiheadAttention(embed_dim,
num_heads, batch_first=True)
        self.norm = torch.nn.LayerNorm(embed_dim)
    def forward(self, text_embeds, image_embeds):
        output = self.multihead_attention(
            query=text_embeds,
            key=image_embeds,
            value=image_embeds
        )[0]
        output = self.norm(output + text_embeds)
        return output
```

Figure 9. Architecture of the CrossAttentionFusion Class

Lastly, combining the image encoder and cross attention layer, we have built the Multimodal LLM which consists of forward and generate functions. Forward function will run the model and return the logits. On the other hand, generate function will generate the

text. The whole architecture of multimodal modal can be examined in Figure 10, Figure 11, and Figure 12.

```
class MultimodalModel(torch.nn.Module):
    def __init__(self, text_model, hyperparams):
        super(MultimodalModel, self).__init__()
        self.hp = hyperparams
        self.text_model = text_model
        self.image_encoder = ImageEncoderVIT()
        for param in self.image_encoder.parameters():
            param.requires_grad = self.hp.get('train_image_encoder',
False)
        self.image_projection = nn.Sequential(
            nn.Linear(self.hp['vision_dim'], self.hp['vision_dim']),
            nn.GELU(),
            nn.Linear(self.hp['vision_dim'], self.hp['text_dim'])
        )
        self.cross_attention  = CrossAttentionFusion(self.hp['text_dim'],
self.hp['num_heads'])
```

Figure 10. Architecture of the Init Function of the Modal

```
def forward(self, image_input, input_ids):
        if self.hp['use_preprocessed']:
            image_features = image_input
        else:
            with torch.no_grad() if not self.hp.get('train_image_encoder',
False) else nullcontext():
                image_features = self.image_encoder(image_input)
        image_features = self.image_projection(image_features)
        text_embeds = self.text_model.get_input_embeddings()(input_ids)
        cross_text_embeds  = self.cross_attention(text_embeds,
image_features
        hidden_states =
self.text_model.model.forward(inputs_embeds=cross_text_embeds)[0]
        logits = self.text_model.lm_head(hidden_states)
        return logits
```

Figure 11. Architecture of the Forward Function of the Modal

```
    def generate(self, tokenizer, image_path, question):
        self.eval()
        self.to(device)

        image = Image.open(image_path).convert('RGB')

        with torch.no_grad():
            image_features = self.image_encoder(image)

        image_features = image_features.unsqueeze(0)

        prompt = prompt_template(question, training=False)
        input_ids = tokenizer.encode(prompt,
return_tensors='pt').to(device)

        generated_ids = input_ids
        max_length_to_loop = self.hp['max_length'] - input_ids.shape[1]

        for _ in range(max_length_to_loop):

            with torch.no_grad():
                logits = self.forward(image_input=image_features,
input_ids=generated_ids)

            next_token_logits = logits[:, -1, :]
            next_token = torch.argmax(next_token_logits, dim=-
1).unsqueeze(-1)

            generated_ids = torch.cat([generated_ids, next_token], dim=-1)

            if next_token.item() == tokenizer.eos_token_id:
                break

        generated_text = tokenizer.decode(generated_ids.squeeze(),
skip_special_tokens=False)

        return generated_text
```

Figure 12. Architecture of the Generate Function of the Modal

We have not just created a single modal architecture but many model architectures. As an example, Figure 13 is one of those different approaches that we have developed. What we have done differently in this model is that, instead of using a single-directional cross-attention mechanism as in the first approach, we introduced a bidirectional cross-attention module. This allows both image features to attend to text features *and* vice versa, promoting deeper interaction and fusion between the modalities. Additionally, we made use of layer-wise image feature processing, where the Vision Transformer (ViT) encoder layers are explicitly iterated through, allowing finer control over image representation extraction. This second architecture also incorporates optional feed-forward networks (FFNs) after attention blocks, which can be activated depending on hyperparameters to further refine modality-specific embeddings. Finally, this model supports layer-level multimodal fusion through multiple stacked bidirectional attention layers, leading to a more sophisticated and flexible integration of visual and textual information. Finally, the whole architecture of the image encoder, cross attention and the modal can be seen in Figure 13, Figure 14, Figure 15, Figure 16, Figure 17 and Figure 18.

```
class ImageEncoderVIT(nn.Module):
    def __init__(self, vit_model):
        super().__init__()
        self.vit_model = vit_model
        self.transform_func =
ViT_B_16_Weights.IMAGENET1K_SWAG_LINEAR_V1.transforms()

    def forward(self, x):
        x = self.transform_func(x)
        x = x.unsqueeze(0).to(device)

        if hyperparams['use_bloat16']:
            x = x.to(torch.bfloat16)

        x = self.vit_model._process_input(x)

        for e_block in self.vit_model.encoder.layers:
            x = e_block(x)

        return x
```

Figure 13. Architecture of the ImageEncoderVIT for the Second Approach

```python
class BidirectionalCrossAttention(nn.Module):
    def __init__(self, embed_dim, num_heads, hyperparams):
        super().__init__()
        self.hp = hyperparams
        self.text_to_image = nn.MultiheadAttention(
            embed_dim=embed_dim,
            num_heads=num_heads,
            batch_first=True
        )
        self.image_to_text = nn.MultiheadAttention(
            embed_dim=embed_dim,
            num_heads=num_heads,
            batch_first=True
        )
        if self.hp['use_cross_ffn']:
            self.text_ffn = nn.Sequential(
                nn.Linear(embed_dim, self.hp['cross_ffn_dim']),
                nn.GELU() if self.hp['cross_ffn_activation'] == 'gelu'
else nn.ReLU(),
                nn.Dropout(self.hp['cross_ffn_dropout']),
                nn.Linear(self.hp['cross_ffn_dim'], embed_dim)
            )
            self.image_ffn = nn.Sequential(
                nn.Linear(embed_dim, self.hp['cross_ffn_dim']),
                nn.GELU() if self.hp['cross_ffn_activation'] == 'gelu'
else nn.ReLU(),
                nn.Dropout(self.hp['cross_ffn_dropout']),
                nn.Linear(self.hp['cross_ffn_dim'], embed_dim)
            )
        else:
            self.text_ffn = nn.Identity()
            self.image_ffn = nn.Identity()
        self.norm1 = nn.LayerNorm(embed_dim)
        self.norm2 = nn.LayerNorm(embed_dim) if self.hp['use_cross_ffn']
else nn.Identity()
```

Figure 14. Architecture of the Cross Attention for the Second Approach

```
    def forward(self, text, image):
        image_attn, _ = self.image_to_text(image, text, text)
        text_attn, _ = self.text_to_image(text, image_attn, image_attn)

        text = self.norm1(text + text_attn)
        image = self.norm1(image + image_attn)

        if self.hp['use_cross_ffn']:
            text = self.norm2(text + self.text_ffn(text))
            image = self.norm2(image + self.image_ffn(image))

        return text, image
```

Figure 15. Architecture of the Forward Function in Cross Attention for the Second Approach

```
class MultimodalModelWithLayerFusion(nn.Module):
    def __init__(self, text_model, hyperparameters):
        super().__init__()
        self.hp = hyperparameters

        self.image_encoder = ImageEncoderVIT(self.hp['vit_model'])
        for param in self.image_encoder.parameters():
            param.requires_grad = self.hp.get('train_image_encoder',
False)

        self.image_layer_proj = nn.Sequential(
            nn.Linear(self.hp['vision_dim'], self.hp['vision_dim']),
            nn.GELU(),
            nn.Linear(self.hp['vision_dim'], self.hp['text_dim'])
        )
        self.text_model = text_model

        self.cross_attentions = nn.ModuleList([
            BidirectionalCrossAttention(
                embed_dim=self.hp['text_dim'],
                num_heads=self.hp['num_heads'],
                hyperparams=self.hp
            ) for _ in range(self.hp['num_cross_layers'])
        ])

        self.image_norm = nn.LayerNorm(self.hp['text_dim'])

        self.use_preprocessed = self.hp.get('use_preprocessed', False)
```

Figure 16. Architecture of the Init Function of the Modal for the Second Approach

```
    def forward(self, image_input, input_ids):
        """
        Args:
            image_input: Either raw images (tensor) or tuple of
(last_layer, all_layers)
            input_ids: Text token IDs
        """
        if self.use_preprocessed:
            last_layer = image_input
        else:
            with torch.no_grad() if not self.hp.get('train_image_encoder',
False) else nullcontext():
                last_layer = self.image_encoder(image_input)
        image_embeds = self.image_layer_proj(last_layer)
        image_embeds = self.image_norm(image_embeds)

        text_embeds = self.text_model.get_input_embeddings()(input_ids)

        for cross_attention in self.cross_attentions:
            text_embeds, image_embeds = cross_attention(text_embeds,
image_embeds)

        hidden_states =
self.text_model.model.forward(inputs_embeds=text_embeds).last_hidden_state
        logits = self.text_model.lm_head(hidden_states)

        return logits
```

Figure 17. Architecture of the Forward Function of the Modal for the Second Approach

```
    def generate(self, tokenizer, image_path, question):
        self.eval()
        self.to(device)

        image = Image.open(image_path).convert('RGB')

        with torch.no_grad():
            image_features = self.image_encoder(image)

        prompt = prompt_template(question, training=False)
        input_ids = tokenizer.encode(prompt,
return_tensors='pt').to(device)

        generated_ids = input_ids
        max_length_to_loop = self.hp['max_length'] - input_ids.shape[1]

        for _ in range(max_length_to_loop):

            with torch.no_grad():
                logits = self.forward(image_input=image_features,
input_ids=generated_ids)

            next_token_logits = logits[:, -1, :]
            next_token = torch.argmax(next_token_logits, dim=-
1).unsqueeze(-1)

            generated_ids = torch.cat([generated_ids, next_token], dim=-1)

            if next_token.item() == tokenizer.eos_token_id:
                break

        generated_text = tokenizer.decode(generated_ids.squeeze(),
skip_special_tokens=False)

        return generated_text
```

Figure 18. Architecture of the Generate Function of the Modal for the Second Approach

We have developed our own prompt structure too. This structure will teach the pre-trained model to know the structure of the conversation. We've added 4 new tokens to the tokenizer as well as padding token as can be seen in Figure 19.

```
def prompt_template(question, answer=None, training=True):
    encoded_prompt = "<start_sequence>"

    encoded_prompt += f"<question>{question}"

    encoded_prompt += f"<answer>"

    if training:
        encoded_prompt += f"{answer}<end_sequence>"

    return encoded_prompt
```

Figure 18. Structure of the Prompt

```
special_tokens_dict = {
    "additional_special_tokens": [
        "<start_sequence>", "<end_sequence>", "<question>", "<answer>",
    ],
    "eos_token": "<end_sequence>",
    "pad_token": "<pad>",
    "bos_token": "<start_sequence>",
}
```

Figure 19. Definition of the Special Tokens

In our first training stages, we used COCO dataset. We made the necessary pre-processing on the dataset. As can be shown in Figure 20, we developed our own dataset class for getting the samples.

```python
class COCOdataset(Dataset):
    def __init__(self, dataframe, tokenizer):
        self.dataframe = dataframe
        self.tokenizer = tokenizer
        self.image_encoder = ImageEncoderVIT()

    def __len__(self):
        return len(self.dataframe)

    def __getitem__(self, index):
        row = self.dataframe.iloc[index]

        question = row['question']
        answer = row['answer']
        image_path = row['image_path']

        image = Image.open(image_path).convert('RGB')
        image_features = self.image_encoder(image)

        tokenized_prompt = self.tokenizer.encode_plus(
            prompt_template(question, answer),
            return_tensors='pt',
            padding='max_length',
            max_length=hyperparams['max_length'],
            truncation=True
        )

        return {'image_features': image_features,
                'tokenized_prompt': tokenized_prompt
                }
```

Figure 20. Implementation of the custom COCOdataset class

APPLICATION DEVELOPMENT

To facilitate user interaction with the VQA model, we have been developing a web-based application including the following components:

- **Frontend:** We are using React.js, with a responsive and user-friendly interface. Features include image upload, question input, and real-time answer rendering.
- **Backend:** We are developing the application with Java Spring Boot, featuring a clean separation of concerns through layered architecture (presentation, application, data access, and model).
- **API Layer:** REST APIs were designed for the communication between frontend and backend.
- **Database:** MongoDB was chosen for its flexible schema and scalability, storing user sessions, questions, images, and predictions.

In order to see if the layers communicate each other successfully, a test-case scenario was performed. For this scenario, the vision encoder ResNet-50 is used. It is important to identify that it is not the actual model but only used for test purposes. After the connection was set between the front-end and back-end, the image and the text were sent to the backend as can be seen in Figure 21.

Figure 21. Main Page of the Web-Based Application

After that, it can be seen that the image and the text were received and transferred between the layers. Finally, they reached to the model layer and the dummy model sent an answer to the dummy question as can be seen in Figure 22.

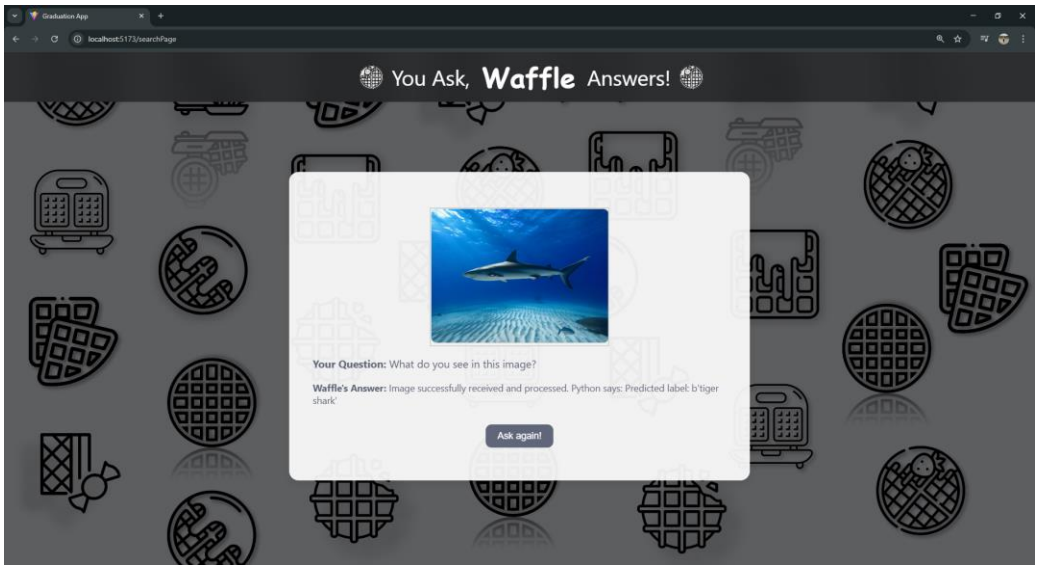


Figure 22. Result Page of the Web-Based Application

The N-tier software architecture can be examined in Figure 23. Presentation represents the Presentation Layer, service represents the Application Layer, and database represents the Data Access Layer. Thanks to this kind of software architecture, the system is modular, scalable, and maintainable. Each layer has its specific responsibilities, making it easier to manage changes, update components, and ensure the overall stability of the system. The separation of concerns also improves the system's flexibility, as different layers can be modified or replaced independently without affecting the entire application.

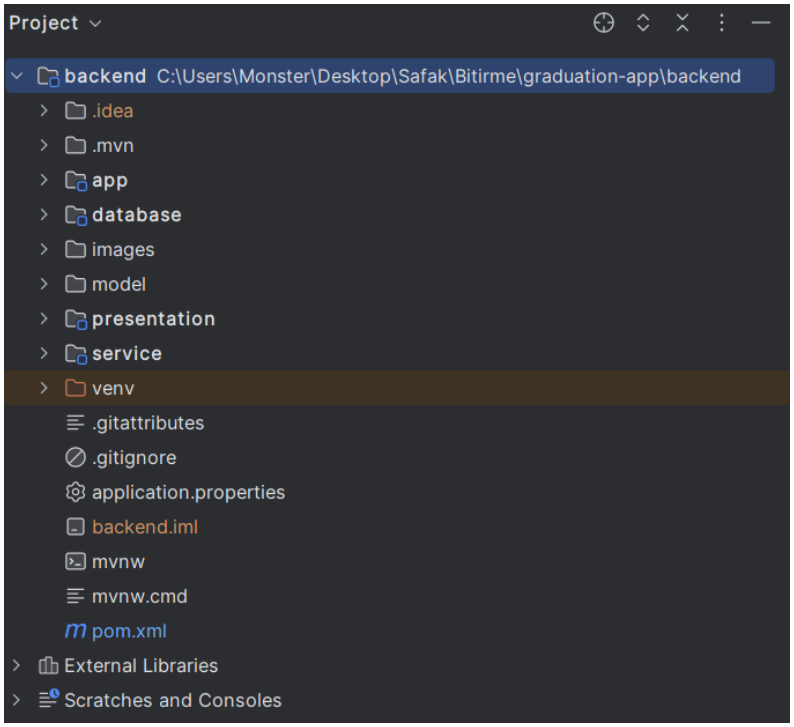


Figure 23. N-tier Software Architecture

A similar kind of architecture was used in the frontend too. The frontend component architecture can be examined in Figure 24. Components represent reusable UI elements, pages combines these components, and configuration files manage the build process. Thanks to this kind of architecture, the system is modular, scalable, and maintainable. Each component has its specific responsibilities, making it easier to manage changes, update UI elements, and ensure the overall consistency of the application. The separation of concerns also improves the system's flexibility, as different components can be modified or replaced independently without affecting the entire frontend.

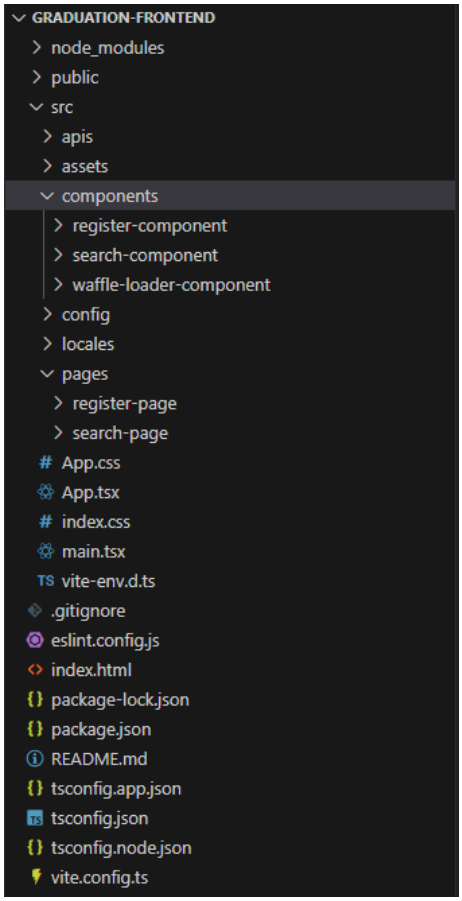


Figure 24. Front-end Software Architecture

The API communication layer establishes a useful interface between the frontend and backend services. This implementation uses Axios for HTTP requests, enabling efficient data exchange through RESTful endpoints. The queryApi function handles both text-based queries and image uploads through FormData, providing a flexible search capability.

By encapsulating API calls in dedicated functions, the architecture achieves clean separation between UI components and data fetching logic. This approach improves maintainability as API changes only require updates in one location. The error handling mechanism ensures proper feedback to users when communication issues occur, enhancing the overall reliability of the application.

```
export const queryApi = async (searchTerm: string, imageFile: File |
null): Promise<SearchResponse> => {
try {
const formData = new FormData();
formData.append("text", searchTerm);

if (imageFile) {
formData.append("image", imageFile);
}

const response = await
axios.post<QueryResponse>('http://localhost:8080/api/query, formData, {
headers: {
'Content-Type': 'multipart/form-data'
}
});

return response.data;
} catch (error) {

throw new Error('Failed to perform query);
}
};
```

Figure 25. QueryApi Class Implementation

CHALLENGES

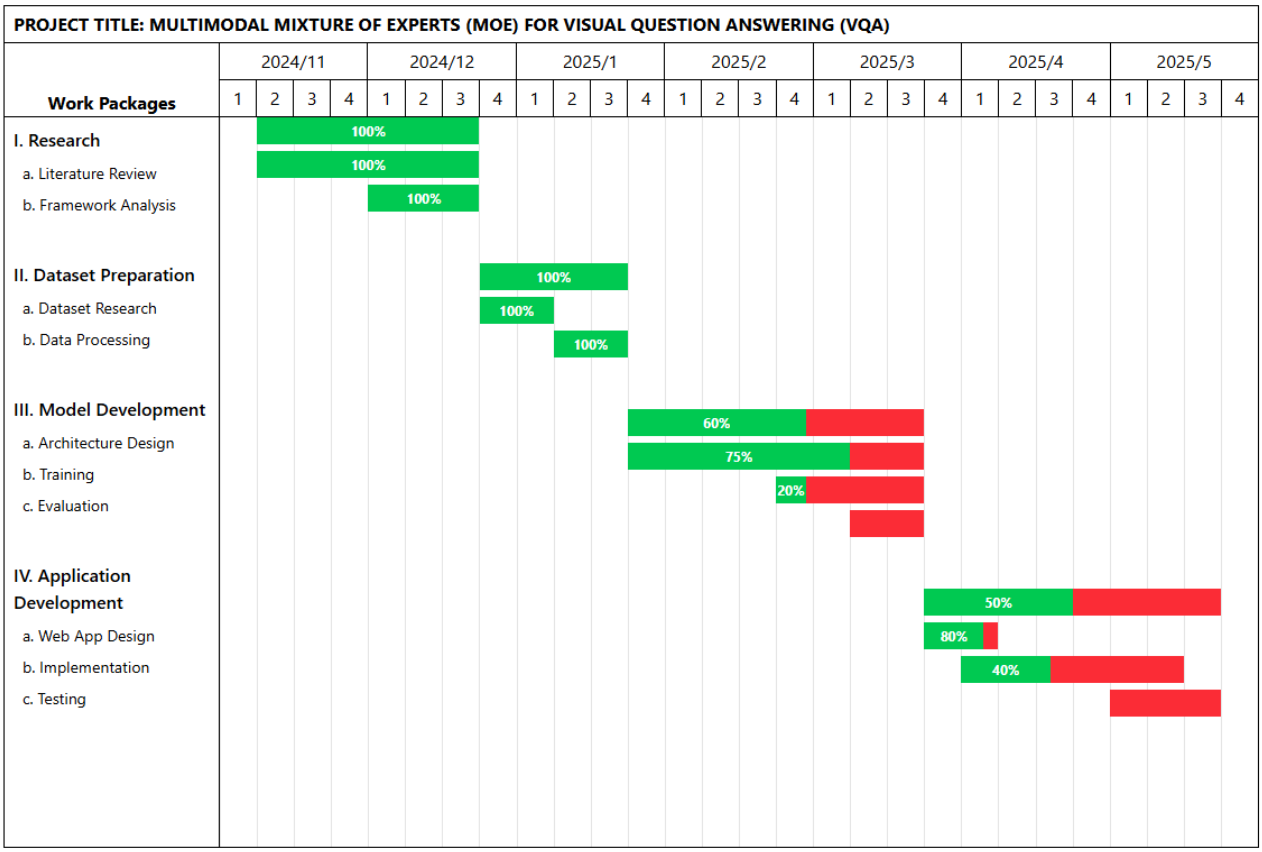
Challenges can be categorized into 5 categories as follows:

- **Computational Limitations**
  - Training our model on Google Colab [36] or Kaggle Notebooks [37] often resulted in crashes or termination due to resource restrictions
- **Hardware Constraints**
  - Our personal computers were not sufficient enough for extensive model training
- **Dataset Size**
  - Large datasets, especially the ones containing images, were difficult to download and process because of storage and bandwidth limitations
- **Model Selection**
  - Finding compact LLMs under 1B parameters with Mixture of Experts (MoE) architecture was challenging, so it limits our experimentation options
- **Project Complexity**
  - Developing a high-performing multimodal question-answering system is complex and requires extensive optimization that even huge high-tech companies struggle with.
- **Latency Issues**
  - Our application currently facing with delays in generating responses due to model inference times

PLAN FOR NEXT PERIOD

Our primary focus for the upcoming period will be continuing the model training process with more diverse datasets to enhance overall performance. We will dedicate significant research efforts toward optimizing Qwen2-0.5B, specifically addressing the challenges presented by its compact Mixture of Experts architecture. Should Qwen2-0.5B continue to underperform despite our optimization attempts, we'll pivot to a non-MoE model that offers better efficiency for our specific use case. Simultaneously, we'll work toward finalizing the development and deployment of our web-based application to ensure seamless user experience. To address our current hardware constraints, we'll investigate cloud-based solutions that could potentially improve both our training and inference capabilities, allowing us to scale more effectively.

PROJECT CALENDAR



EXPENDITURES:

	Work Packages				
	I	**II**	**III**	**IV**	**TOTAL**
Equipment	-	-	-	-	-
Consumable goods	-	-	-	-	-
Publications/Software	-	-	-	-	-
Transportation	-	-	-	-	-
Services	-	-	-	-	-
TOTAL	-	-	-	-	-

Equipment:
-

Consumable goods:
-

Publications/Software:

- Python [38], PyTorch [39], JupyterLab [40], Java Spring Boot, REST API, React, MongoDB

Transportation:
-

Services:

- Google Colab, Google Cloud and Kaggle Notebooks

REFERENCES:

[1] Agrawal *et al.*, "VQA: Visual question answering," *International Journal of Computer Vision*, vol. 123, no. 1, pp. 4–31, Nov. 2016. doi:10.1007/s11263-016-0966-6

[2] J. Zhang, L. Zheng, M. Wang, and D. Guo, "Training a small emotional vision language model for visual art comprehension," Lecture Notes in Computer Science, pp. 397–413, Nov. 2024. doi:10.1007/978-3-031-72855-6_23

[3] A. Vaswani et al., 'Attention is All you Need', in Advances in Neural Information Processing Systems, 2017, vol. 30.

[4] K. Han *et al.*, "A survey on Vision Transformer," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 45, no. 1, pp. 87–110, Jan. 2023. doi:10.1109/tpami.2022.3152247

[5] L. Beyer et al., 'PaliGemma: A versatile 3B VLM for transfer', ArXiv, vol. abs/2407.07726, 2024.

[6] D. Zhang, R. Nayak, and M. A. Bashar, "Exploring fusion strategies in deep learning models for multi-modal classification," *Communications in Computer and Information Science*, pp. 102–117, 2021. doi:10.1007/978-981-16-8531-6_8

[7] Y. Goyal, T. Khot, D. Summers-Stay, D. Batra, and D. Parikh, 'Making the V in VQA Matter: Elevating the Role of Image Understanding in Visual Question Answering', in Conference on Computer Vision and Pattern Recognition (CVPR), 2017.

[8] T.-Y. Lin *et al.*, "Microsoft Coco: Common Objects in Context," *Lecture Notes in Computer Science*, pp. 740–755, 2014. doi:10.1007/978-3-319-10602-1_48

[9] A. Singh *et al.*, "Towards VQA models that can read," *2019 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 8309–8318, Jun. 2019. doi:10.1109/cvpr.2019.00851

[10] C. Janiesch, P. Zschech, and K. Heinrich, "Machine learning and deep learning," *Electronic Markets*, vol. 31, no. 3, pp. 685–695, Apr. 2021. doi:10.1007/s12525-021-00475-2

[11] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding,” *Proceedings of the 2019 Conference of the North*, pp. 4171–4186, Jun. 2019. doi:10.18653/v1/n19-1423

[12] A. Dosovitskiy et al., ‘An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale’, *ArXiv*, vol. abs/2010.11929, 2020.

[13] A. Chaoub, C. Cerisara, A. Voisin, and B. Iung, “Towards interpreting deep learning models for industry 4.0 with gated mixture of experts,” *2022 30th European Signal Processing Conference (EUSIPCO)*, pp. 1412–1416, Aug. 2022. doi:10.23919/eusipco55093.2022.9909884

[14] D. A. Hudson and C. D. Manning, ‘Gqa: A new dataset for real-world visual reasoning and compositional question answering’, in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2019, pp. 6700–6709.

[15] H. Laurençon, L. Tronchon, M. Cord, and V. Sanh, ‘What matters when building vision-language models?’, in The Thirty-eighth Annual Conference on Neural Information Processing Systems, 2024.

[16] X. Zhai, B. Mustafa, A. Kolesnikov, and L. Beyer, “Sigmoid loss for language image pre-training,” *2023 IEEE/CVF International Conference on Computer Vision (ICCV)*, Oct. 2023. doi:10.1109/iccv51070.2023.01100

[17] O. Tolstikhin et al., ‘MLP-Mixer: An all-MLP Architecture for Vision’, ArXiv, vol. abs/2105.01601, 2021.

[18] P. Gao *et al.*, “Clip-adapter: Better Vision-language models with feature adapters,” *International Journal of Computer Vision*, vol. 132, no. 2, pp. 581–595, Sep. 2023. doi:10.1007/s11263-023-01891-x

[19] Hugo Laurençon, undefined., et al. "Building and better understanding vision-language models: insights and future directions," in *ArXiv*, vol. abs/2408.12637, 2024.

[20] “Squad2.0,” The Stanford Question Answering Dataset, https://rajpurkar.github.io/SQuAD-explorer/ (accessed Apr. 2, 2025).

[21] Triviaqa, https://nlp.cs.washington.edu/triviaqa/ (accessed Apr. 2, 2025).

[22] "Sample datasets," Databricks Documentation, https://docs.databricks.com/aws/en/discover/databricks-datasets (accessed Apr. 2, 2025).

[23] "Common objects in context," COCO, https://cocodataset.org/#home (accessed Apr. 2, 2025).

[24] "Visual reasoning in the real world," GQA: Visual Reasoning in the Real World, https://cs.stanford.edu/people/dorarad/gqa/index.html (accessed Apr. 2, 2025).

[25] "HuggingFaceTB/SMOLLM2-135m · hugging face," HuggingFaceTB/SmolLM2-135M · Hugging Face, https://huggingface.co/HuggingFaceTB/SmolLM2-135M (accessed Apr. 2, 2025).

[26] "HuggingFaceTB/SMOLLM2-360m · hugging face," HuggingFaceTB/SmolLM2-360M · Hugging Face, https://huggingface.co/HuggingFaceTB/SmolLM2-360M (accessed Apr. 2, 2025).

[27] "Openai-community/GPT2 · hugging face," openai-community/gpt2 · Hugging Face, https://huggingface.co/openai-community/gpt2 (accessed Apr. 2, 2025).

[28] "Qwen/QWEN2-0.5B · hugging face," Qwen/Qwen2-0.5B · Hugging Face, https://huggingface.co/Qwen/Qwen2-0.5B (accessed Apr. 2, 2025).

[29] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," arXiv.org, https://arxiv.org/abs/1512.03385v1 (accessed Apr. 2, 2025).

[30] M. Tan and Q. V. Le, "EfficientNet: Rethinking model scaling for Convolutional Neural Networks," arXiv.org, https://arxiv.org/abs/1905.11946v5 (accessed Apr. 2, 2025).

[31] A. Dosovitskiy et al., "An image is worth 16x16 words: Transformers for image recognition at scale," arXiv.org, https://arxiv.org/abs/2010.11929v2 (accessed Apr. 2, 2025).

[32] "React," React Blog RSS, https://react.dev/ (accessed Apr. 2, 2025).

[33] "Spring boot3.4.4," Spring Boot, https://spring.io/projects/spring-boot (accessed Apr. 2, 2025).

[34] Ibm, "What is a rest api?," IBM, https://www.ibm.com/think/topics/rest-apis#:~:text=A%20REST%20API%20(also%20called,transfer%20(REST)%20architectural%20style. (accessed Apr. 2, 2025).

[35] "The world's leading Modern Database," MongoDB, https://www.mongodb.com/ (accessed Apr. 2, 2025).

[36] Google colab, https://colab.research.google.com/ (accessed Apr. 2, 2025).

[37] "Run Data Science & Machine Learning Code Online," Kaggle, https://www.kaggle.com/code (accessed Apr. 2, 2025).

[38] "Welcome to Python.org," Python.org, https://www.python.org/ (accessed Apr. 2, 2025).

[39] "Get started," PyTorch, https://pytorch.org/ (accessed Apr. 2, 2025).

[40] "Project jupyter," Project Jupyter, https://jupyter.org/ (accessed Apr. 2, 2025).